GitHub SSH Key Bootstrap Script 🔐🐙

Open source software - by Paulo Jerônimo

Bootstrap a new machine for GitHub SSH access using a GPG-encrypted private key embedded in a self-verifying Bash script.

The script prompts for the GPG passphrase, validates integrity (hash of the script itself and the GPG blob), installs the key at `~/.ssh/id_ed25519_gh`, generates the public key at `~/.ssh/id_ed25519_gh.pub`, and ensures a `Host github.com` entry exists in `~/.ssh/config`.

Read this PDF in HTML format

1 Motivation 🚀

The main goals of this project are:

1. To provide an easy way to bootstrap a new machine for GitHub access using a public SSH key that was already configured in my GitHub account, while retrieving the corresponding private key from a Git repository.
2. To demonstrate some of the skills required to build a project like this, including symmetric and asymmetric cryptography knowledge using both GPG and SSH, and advanced Bash scripting.
3. To demonstrate that, in the current AI-assisted software development era, these skills can be improved significantly with the help of AI agents working alongside the developer.

To achieve that, I used Bash, GPG, and SSH to build a security-focused script that I can execute with a short sequence of commands:

```
read -p "USER_REPO: " USER_REPO
SETUP_URL="https://raw.githubusercontent.com/${USER_REPO}/refs/heads/main/setup.sh"
curl -fsSL "$SETUP_URL" -O
bash ./setup.sh
```

Note: the script validates its own integrity, so it must run from a local file (temporary is fine) instead of `curl ... | bash` or `bash <(curl ...)`.

After running the script, validate that SSH access to GitHub works:

```
git clone git@github.com:${USER_REPO}.git
```

Optional checks for the files/configuration created by the script:

```
ls -l ~/.ssh/id_ed25519_gh*
cat ~/.ssh/config
```

Important: this workflow is useful and practical, but the project still has an open authenticity gap for published scripts (documented below in TODO (Authenticity Verification)) ⚠️

Click this link to view an animated GIF with the flow presented in this section.

2 Development model 🏛️

This project is developed using a "Cathedral" model of open source development.

In practice, this means I maintain two repositories:

- a private repository, where development happens with granular commits, experiments, and auxiliary scripts used for development workflows (for example, sync scripts)
- a public repository (the one users normally see on GitHub), used mostly to publish releases

This project is developed by one person (Paulo Jeronimo) and AI agents.

Even though the development is led by a single person, contributions are welcome through GitHub pull requests.

3 What the setup script does ✅

Running `./setup.sh` does all of the following:

- decrypts the embedded private key using your GPG symmetric passphrase 🔓
- installs the GitHub key at `~/.ssh/id_ed25519_gh` 🔑
- generates `~/.ssh/id_ed25519_gh.pub` 🧾
- creates (or ensures) the recommended `Host github.com` block in `~/.ssh/config` ⚙️

The `~/.ssh/config` example shown in SSH key workflow (recommended) is not just a recommendation. The script itself writes that GitHub block for you.

Test locally 🧪:

```
./setup.sh
```

If `~/.ssh/id_ed25519_gh` already exists, the script aborts by default. Use `--overwrite` to replace the key and rewrite the `Host github.com` block in `~/.ssh/config`:

```
./setup.sh --overwrite
```

(Optional) Use `DEBUG` to print integrity details 🐞:

```
DEBUG= ./setup.sh
```

4 When the key must be regenerated 🔁

Whenever you change the private key, the GPG block and hashes must be updated in the script.

Use update mode to automate the entire process (generate the blob, replace the block, increment the internal version, and recalculate hashes). It prompts for the GPG passphrase twice for confirmation:

```
./setup.sh --update
```

Example with an explicit key path:

```
./setup.sh --update -k /path/to/id_rsa
```

Options:

- `-k` OpenSSH private key path (optional; if omitted, the value is read from `.env`, `.env.sample`, or the built-in default)
- `-s` script path to update (default: current script path)
- `--no-version-bump` keep `VERSION` unchanged (use when you know the embedded key did not change)
- `--metadata-only` update only `UPDATED_AT` and `SCRIPT_SHA256` (no key/blob/version changes)

4.1 When to use `--no-version-bump`

Use `--no-version-bump` when you are updating the script metadata or re-encrypting the embedded key blob, but you know the underlying SSH private key is the same and you do not want to advance `VERSION`.

Example:

```
./setup.sh --update --no-version-bump -k /path/to/id_rsa -s ./setup.sh
```

Notes:

- `SCRIPT_SHA256` and `BLOB_SHA256` may still change.
- `VERSION` will remain unchanged because the flag explicitly overrides version bump behavior.

4.2 When to use `--metadata-only`

Use `--metadata-only` when you want to refresh only the script metadata timestamp and script hash without touching the embedded SSH key, the encrypted GPG blob, or `VERSION`.

Example:

```
./setup.sh --update --metadata-only -s ./setup.sh
```

Notes:

- `UPDATED_AT` will be refreshed using `date -Is`.
- `SCRIPT_SHA256` will be recalculated.
- `VERSION`, `BLOB_SHA256`, and `KEY_PUBLIC_SHA256` remain unchanged.

Important: if the GPG symmetric passphrase is forgotten, the embedded private key in `setup.sh` cannot be recovered. The operational recovery path is to use the original private key material (or generate a new key) and run `./setup.sh --update` to generate a new `setup.sh` with a new GPG passphrase. See also Security notes and TODO (Authenticity Verification). 🚨

4.3 After manual changes to `setup.sh`

If you manually edit `setup.sh` and then run it, you may see:

```
Error: script integrity check failed (hash mismatch).
```

This happens because `setup.sh` validates its own content with `SCRIPT_SHA256`. Any manual edit changes the file hash.

Recommended path (most cases):

1. Run `./setup.sh --update -k /path/to/private_key -s ./setup.sh` to refresh the embedded blob and hashes automatically.
2. If you know the embedded SSH key did not change and you do not want to increment `VERSION`, use `./setup.sh --update --no-version-bump -k /path/to/private_key -s ./setup.sh`.
3. If you only need to refresh script metadata (timestamp + script hash), use `./setup.sh --update --metadata-only -s ./setup.sh`.

Manual hash recomputation is a fallback/troubleshooting procedure (not the primary workflow). It is still useful when:

- you changed only script text/logic and do not want to run update mode
- you are debugging a hash mismatch
- update mode is temporarily broken and you need to recover the script manually
- you want to audit the calculated values directly

If you need the manual procedure, compute the current hashes locally:

```
awk 'BEGIN{skip=0} {if ($0 ~ /^SCRIPT_SHA256=/) next} {print}' setup.sh | sha256sum
awk 'BEGIN{f=0} /^-----BEGIN PGP MESSAGE-----$/{f=1} {if(f) print} /^-----END PGP MESSAGE-----$/
        {exit}' setup.sh | sha256sum
```

If your system does not have `sha256sum`, use `shasum -a 256` instead.

After updating the hash value(s) in `setup.sh`, run `./setup.sh` again.

5 Script dependencies 🧰

External utilities used internally by `setup.sh`:

- `bash` (script runtime)
- `gpg` (required for install mode and update mode)
- `ssh-keygen` (required for install mode and update mode key validation/fingerprint generation)
- `sha256sum` or `shasum -a 256` (integrity and hash generation)
- `awk` (hash extraction, block replacement, config edits)
- `sed` (metadata updates such as `VERSION` and hashes in update mode)
- `grep` (key and armored block validation)
- `tr` (line ending normalization)
- `mktemp` (temporary files)
- `mkdir`, `mv`, `chmod`, `cat` (file and permission management)
- `head`, `tail`, `wc` (debug output paths when `DEBUG` is enabled)

Notes:

- The script checks that Bash 5 or newer is available and aborts if the version is unsupported.
- The script checks for `gpg`, `ssh-keygen`, and a SHA-256 command (`sha256sum` or `shasum`) explicitly.
- Some utilities are only exercised in specific modes (`install`, `--update`) or only in debug/error paths.

6 Tested environments ✅

The script has been tested in the following environments:

- Ubuntu 25.10

- Termux on Android

If the script is executed in an environment different from the tested ones above, it prints a warning but does not abort execution.

7 SSH key workflow (recommended) 🧭

Use separate keys for GitHub and local servers, and let SSH select the correct key automatically by host.

Example `~/.ssh/config`:

```
Host github.com
  HostName github.com
  User git
  IdentityFile ~/.ssh/id_ed25519_gh
  IdentitiesOnly yes
  AddKeysToAgent yes

Host *.local my-server
  User your-user
  IdentityFile ~/.ssh/id_ed25519_servers
  IdentitiesOnly yes
  AddKeysToAgent yes
```

With this setup:

- `git clone git@github.com:org/repo.git` automatically uses the GitHub key.
- `ssh my-server` automatically uses the server key.

The `setup.sh` script automatically writes the `Host github.com` block shown above. You only need to add your server-specific blocks (for example `*.local` or `my-server`). For local validation of the update workflow, see Non-interactive tests.

8 Non-interactive tests 🤖

Configure the environment file and run the test script:

```
cp test/.env.sample test/.env
```

Generate the example key used by `test/.env.sample` (`KEY_PATH=~/.ssh/id_ed25519_gh`):

```
ssh-keygen -t ed25519 -f ~/.ssh/id_ed25519_gh -C "github-key" -N ""
```

Edit `test/.env` and set:

- `KEY_PATH` OpenSSH private key path
- `GPG_PASSPHRASE` passphrase for encryption
- `SCRIPT_PATH` (optional) script path to update (default: `./setup.sh`)

Run:

```
./test/setup.sh
```

Note: if `KEY_PATH` does not exist, the test script automatically generates a key with `ssh-keygen`.

9 Security notes 🛡️

1. This script does not require execution as `root`. ALWAYS, ALWAYS, ALWAYS be very careful when running any script on your machine. Read scripts carefully before executing them, especially if they contain commands prefixed with `sudo`.
2. The script can be public, but the GPG passphrase must be strong and private.
3. Only run downloaded scripts from a source you control.
4. A separate GPG signature mechanism is still needed to validate the authenticity of the published script (see TODO (Authenticity Verification)).

10 TODO (Authenticity Verification) 📌

Current real risks in this project:

- The script verifies integrity (`SCRIPT_SHA256` and `BLOB_SHA256`) but not publisher authenticity. ⚠️
- If an attacker controls the distribution channel (for example: compromised hosting, poisoned URL, or a malicious mirror), they can publish a modified script together with new hashes and a different encrypted blob. ⚠️
- This matters especially when users execute a downloaded script before manually inspecting it. ⚠️

How the TODO items below reduce those risks:

- Detached signature verification adds publisher authenticity checks before decryption. ✅
- Publishing the signature separately allows independent distribution and verification. ✅
- Using a public key from a trusted profile (Keybase) makes the expected signer explicit. ✅
- Optional signature hash pinning reduces risk of silent signature replacement in transit. ✅

Goal: validate script authenticity via a detached `.asc` GPG signature using the public key published on Keybase.

Planned steps:

1. Publish the script signature Generate locally: `gpg --armor --detach-sign setup.sh` Publish the generated file (for example: `setup.sh.asc`) at a public URL.
2. Define URLs used by the script `SIG_URL`: public URL of the detached signature `PUBKEY_URL`: see Keybase public key URL
3. Add verification to the script before any decryption Import the public key: `curl -fsSL "$PUBKEY_URL" | gpg --import >/dev/null` Download the signature: `curl -fsSL "$SIG_URL" -o /tmp/setup.sh.asc` Verify the signature: `gpg --verify /tmp/setup.sh.asc "$0"` Abort if verification fails.
4. (Optional) Pin the SHA-256 hash of the signature Compute the hash locally and embed it in the script. Validate the hash after downloading the `.asc`.
5. End-to-end test Publish the script and its signature. Execute after downloading to a local file (using the URL pattern shown in Setup script raw download URL). For example: `curl -fsSL "$SETUP_UeL" -O && bash ./setup.sh`. Confirm it fails if the script is modified.

11 References 📚

1. Setup script raw download URL (example for this project):
   https://raw.githubusercontent.com/paulojeronimo/setup-github-ssh-key/refs/heads/main/setup.sh

2. The Cathedral and the Bazaar:
   https://en.wikipedia.org/wiki/The_Cathedral_and_the_Bazaar

3. Keybase public key URL (paulojeronimo):
   https://keybase.io/paulojeronimo/pgp_keys.asc

4. GitHub Docs: Generating a new SSH key and adding it to the ssh-agent (`ed25519` as the recommended default and `RSA` as a legacy fallback):
   https://docs.github.com/en/authentication/connecting-to-github-with-ssh/generating-a-new-ssh-key-and-adding-it-to-the-ssh-agent?platform=linux

5. OpenSSH documentation (manual pages index):
   https://www.openssh.com/manual.html

6. GnuPG documentation:
   https://gnupg.org/documentation/

7. GNU Bash manual:
   https://www.gnu.org/software/bash/manual/

8. GNU Awk (gawk) manual:
   https://www.gnu.org/software/gawk/manual/